第 7-1 讲 · 从零设计一个小软件

> **模块 7 · 独立造东西** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成模块 0–6

本节目标

读完并做完这一讲，你能：

1. **独立把一个模糊的想法变成可执行的方案**（需求 → 数据 → 界面 → 技术选型）。
2. 掌握**四步设计法**，并知道每一步要产出什么。
3. 学会**技术选型**：什么时候用桌面应用、什么时候用网页、什么时候用命令行。
4. 理解"**最小可用版本（MVP）**"的含义，以及为什么它决定你能不能做完。
5. **产出一份自己的设计方案**（本讲的动手任务）。

一、先承认一件事

1.1 你现在会的东西比你以为的多

回顾这 24 讲你学到的（这不是知识清单，是**能力清单**）：

你学过的	它能用来做什么
终端、文件、路径（模块 0–1）	操作任何开发环境
主进程 / 渲染进程 / IPC（模块 2）	**理解任何 Electron 应用的结构**
数据库、迁移、软删除（模块 2–5）	**设计任何需要存数据的应用**
读代码心法（模块 3）	看懂**任何**陌生项目
改代码流程（模块 4）	在**任何**项目里安全地加功能
验证与安全（模块 5）	保证质量、保护用户数据
打包与发布（模块 6）	把软件交给**任何人**

关键认知：

> **这些知识里，"记账簿"只占很小一部分。**
> **主进程/渲染进程、IPC、数据库设计、验证、打包——把"记账"换成"记笔记""管书签""统计健身数据"，整套流程一模一样。**

模块 7 要做的就是这件事：把"记账簿"这三个字从你的知识里摘掉，看看剩下什么。

剩下的就是"能独立开发软件"的能力。

1.2 一个残酷但重要的提醒

"能独立开发"不等于"什么都能做"。 它意味着：

能	不能
独立设计并实现一个中小型应用	一个人做出微信（那是几千人的活）
判断一个需求可不可行、大概多久	准确预估大型项目的工期
遇到不懂的东西知道怎么查、怎么试	记住所有 API（没人记得住）
把软件交付给用户	保证它一定能赚钱

"知道自己不能做什么"也是能力的一部分。

二、四步设计法

2.1 全景

任何软件的设计，都走这四步：

① 想清楚"到底要什么" → 需求定义
② 想清楚"数据长什么样" → 数据模型
③ 想清楚"用户怎么用" → 界面与流程
④ 想清楚"用什么技术" → 技术选型

顺序不能乱：**先有数据，才有界面**；**先有需求，才有数据。**

新手最常犯的错：**从界面开始想**（"我要一个漂亮的表格"），结果做一半发现数据存不下想要的字段。

2.2 现实比喻：盖房子

步骤	盖房子	做软件
① 需求	几口人住？要不要车库？	谁用？解决什么问题？
② 数据模型	**地基和承重结构**	数据表与字段
③ 界面流程	房间怎么连、门开在哪	用户点几次能完成任务
④ 技术选型	砖混还是钢架	桌面 / 网页 / 命令行

注意 ② 的位置：**它是地基。**地基错了，上面怎么改都别扭。

这就是为什么模块 2-3 花一整讲讲数据库——数据模型是软件的地基。

三、第一步：需求定义

3.1 三个问题（4-2 讲原样复用）

问题	作用
① 要什么?	具体表现，最好能用一句话说清
② 影响谁?	谁用、在哪用、什么时候用
③ 边界在哪?	**明确不做什么**（防范围蔓延）

3.2 一个好需求的样子

对比一下：

差的需求	好的需求
"我要一个记账软件"	"我要记录每天的开销，能按月看总数，数据存在我自己电脑上"
"要好看"	"深色模式、列表清晰、手机上也能看"
"要能搜索"	"能按备注关键词搜，也能按日期范围筛"

差别在哪：**好的需求可以被验证**（"能按月看总数"——做完就能确认对不对）。

"不好验证的需求"是项目杀手——因为永远做不完。

3.3 一个实用技巧：写下"验收标准"

给每条需求配一句"怎么算做完了"：

```
需求：能按月看总数
验收：选 9 月，能看到这个月的收入合计、支出合计、结余


需求：数据存在自己电脑上
验收：断网能用；数据目录里有一个数据库文件；用别的电脑打开读不到
```

这个习惯的价值：**它让你在做的过程中随时知道"还差什么"**，而不是做到最后才发现理解错了。

四、第二步：数据模型（最重要的一步）

4.1 问三个问题

问题	例子（记账簿的真实答案）
要存哪些"东西"？	账目、账户、分类、预算、周期规则……
每个"东西"有哪些属性？	账目：日期、金额、类型、账户、分类、备注、标签……
它们之间什么关系？	一笔账属于一个账户、一个分类

这三问的答案，就是数据库的"表 + 字段 + 外键"。

4.2 三个必须提前想清楚的坑

这些坑我在记账簿里都踩过或专门绕开了，你可以直接借用经验：

坑一：金额用什么类型

我实测过的证据（2-3 讲）：

```
8.7 * 100        // 869.9999999999999   ← 本该是 870!
0.1 + 0.2        // 0.30000000000000004
```

结论：涉及钱的字段，一律用"最小货币单位的整数"（存"分"，不存"元"）。

这条经验适用于任何涉及钱、涉及精确计数的场景。

坑二：删除要不要真删

记账簿的选择：软删除（标记 `deleted_at`，数据留着）。

问自己三个问题：

问题	如果答案是"是" → 用软删除
用户会不会误删?	会（几乎所有软件都是）
删掉的数据以后可能有用吗?	可能（对账、追溯）
需要一个"回收站"吗?	要

代价：文件会变大。**收益**：误删能救回来。

坑三：时间怎么存

记账簿的做法（2-3 讲的真实设计）：

存什么	用什么类型	为什么
业务日期（"这笔账算哪天"）	`TEXT`，格式 `YYYY-MM-DD`	排序、比较可以直接用字符串；**没有时区问题**
操作时间戳（"什么时候创建的"）	`INTEGER`（Unix 毫秒）	精确、便于计算

为什么要分两种：**业务日期不该有时区问题**（用户说"9 月 28 日的账"，就是 9 月 28 日，不该因为换了时区变成 27 日）。

这是一个容易被忽略但很重要的设计。

4.3 检查你的数据模型

做一个"能不能装下需求"的自检：

```
拿出你在第一步写的每条需求，问：
"这个需求需要的数据，我的表里存得下吗？"

例：
需求：按月看总数 → 账目表有 date 字段 ✅ 够了
需求：按分类统计 → 有 category_id ✅ 够了
需求：导出 Excel → 所有字段都够 ✅
需求：显示账户余额 → 需要"账户"表和"初始余额"字段 ✅（记账簿有 initial_balance）
```

如果有需求装不下 → 回去改数据模型（而不是等做到界面时才发现）。

五、第三步：界面与流程

5.1 从"用户要完成什么"出发

不要先想"我要几个页面"，先想**用户要完成哪些事**：

> 用户要做的三件事（记账簿的例子）：
> ① 记一笔账
> ② 看看这个月花了多少
> ③ 找某一笔账

然后再想"每件事点几次能完成"：

任务	记账簿的做法	点击数
记一笔账	首页「+ 新增记录」	1 次点击 + 填表
看本月花多少	打开首页就看到三个数字	**0 次**（直接显示）
找一笔账	搜索框输关键词	1 次输入

"看本月花多少"是 0 次点击——这是**故意设计的**（把最重要的信息放在第一眼看到的地方）。

这个思路叫"以任务为中心"，而不是"以页面为中心"。

5.2 设计流程时问自己

问题	为什么问
最高频的操作是什么?	它应该在**最容易点到**的位置
最容易出错的操作是什么?	它应该**有二次确认**
用户会迷路吗?	需要清晰的导航和"我在哪"的提示
第一次用的人知道该干什么吗?	需要空状态提示（"还没有记录，点这里开始"）

记账簿里这些都有对应：

- 高频操作（新增记录）在**右上角固定位置**
- 危险操作（删除、清空回收站）**都有确认弹窗**
- 侧栏有**选中态高亮**（知道自己在哪）
- 空状态有**引导文案**（"点右上角「选择截图」"）

5.3 一个省时间的技巧

先画"文字版流程"，不画图：

```
新增记录流程：
点「＋」→ 弹窗打开
  → 选类型（支出/收入/转账）
  → 填金额（必填）
  → 选分类（支出/收入必填，转账不需要）
  → 选账户（必填）
  → [转账时] 选转入账户
  → 填备注（可选）
  → 点保存
  → 弹窗关闭 + 列表刷新 + 提示「已保存」
```

写下来之后你会发现很多问题：

- "转账时不需要分类"——这个规则要在界面和校验里都体现（3-2 讲见过）
- "保存后要刷新列表"——这是个技术细节，但不写下来容易忘

文字流程花 5 分钟，能省掉后面半小时的返工。

六、第四步：技术选型

6.1 三个基本选项

形态	适合	例子	代价
桌面应用（Electron）	需要碰本地文件、要离线、要系统集成（托盘/快捷键）	记账簿、VS Code、Obsidian	体积大（100 MB+）、要打包
网页应用	需要多设备访问、要分享给别人	在线笔记、协作工具	**必须有服务器**、数据不在用户手里、要处理登录
命令行工具	自己用、批处理、自动化	备份脚本、数据转换	不好给非技术用户用

6.2 怎么选（三个判断）

问自己三个问题：

问题	如果答案是"是"
需要多设备同步吗?	→ 网页应用（同步必须有服务器）
数据必须留在用户电脑上吗?	→ 桌面应用
要给不懂技术的人用吗?	→ 桌面应用或网页应用（不要命令行）

记账簿的选择逻辑（这是个好的示例）：

```
需要多设备吗？ → 不需要（个人记账，一台电脑够）
数据要留本地吗？ → 需要（隐私是核心卖点）
要给普通人用吗？ → 要（所以不能是命令行）
        ↓
结论：桌面应用（Electron）
```

注意：选 Electron 还有个隐含理由——**记账簿需要数据库、需要 OCR、需要托盘**，这些都是"桌面能力"。**网页应用做不到这些**（浏览器里没有文件系统和托盘）。

6.3 技术栈的选择原则

不要追新，要追"资料多、坑少"：

记账簿的选择	理由
Electron	学一次能用十年，VS Code 也是它
React	资料最多、招人最好招
TypeScript	类型能提前发现错误（3-3 讲见过它的价值）
SQLite	不用装服务器，整个库就是一个文件
Tailwind	不用发明 CSS 类名

一条实用建议：

> **选技术时，先搜"这个技术 + 遇到的问题"，看有多少人问过。**
> 如果搜索结果很多，说明**坑已经有人踩过**，你能搜到答案。**冷门技术的坑只能自己填。**

6.4 别忘了"复用"

记账簿里大量复用了现成的东西：

需求	做法
OCR 识别	用开源的 PP-OCRv4 模型 + onnxruntime（**不自己训模型**）
图表	用 ECharts（**不自己画 SVG**）
Excel 导出	**自己写了一个最小 xlsx 写出器**（因为库太大，而需求很简单）
界面组件	自己写（因为要精确控制视觉和动效）

判断标准：

情况	选择
通用、成熟、无特殊要求	**用现成的库**
需求很简单，而库很大	**自己写**（比如 xlsx 写出器）
是产品的核心差异化	**自己写**（比如界面和动效）

七、动手任务

步骤 1：想出三个你自己的软件点子

要求：不要"记账簿 2.0"，要**你自己真的会用**的东西。

几个参考方向（都适合练手，因为数据模型简单）：

点子	核心数据	适合练什么
读书笔记管理	书名、作者、进度、笔记、标签	增删改查 + 搜索
健身记录	日期、动作、组数、次数、重量	数据统计（进步曲线）
家庭物品清单	物品名、位置、数量、购买日期	分类 + 搜索 + 提醒
学习打卡	日期、科目、时长、心得	日历视图（复用记账簿思路）
书签/链接收藏	标题、网址、标签、备注	导入导出

写下来三个（就写名字和一句话）。

步骤 2：选一个，做需求定义

选一个你最想做的，回答三个问题：

```
① 要什么？
    （一句话说清：它解决什么问题）

② 影响谁？
    （谁用、在哪用、多久用一次）

③ 边界在哪？
    （明确"这一版不做什么"）
```

给每条需求配一句验收标准（3.3 节）。

步骤 3：设计数据模型

问三个问题（4.1 节）：

```
① 要存哪些"东西"？（大概 2–4 张表）
② 每张表有哪些字段？
③ 它们之间什么关系？
```

用这个格式写（模仿记账簿的表结构）：

```
表：items
  id           唯一标识
  name         名称（必填）
  category_id  所属分类 → 指向 categories 表
  created_at   创建时间
  deleted_at   删除时间（软删除）

表：categories
  id
  name
  ...
```

然后自检：我在步骤 2 写的每条需求，这些表装得下吗？

步骤 4：写文字版流程

挑最核心的一个任务（比如"新增一条记录"），用文字把流程写出来：

```
点「新增」→ 打开表单
  → 填 XXX（必填）
  → 选 YYY
  → 点保存
  → 关闭 + 刷新 + 提示
```

检查：

- 有没有"必填"和"可选"的区分？
- 有没有"这种情况下不显示某个字段"的规则？
- 出错时提示什么？

步骤 5：技术选型

回答三个判断问题（6.2 节），写下结论：

```
需要多设备同步吗？
数据必须留本地吗？
要给不懂技术的人用吗？
        ↓
结论：_____（桌面 / 网页 / 命令行）
```

然后写下你要用的技术栈，每条附一句理由。

最后回答一个问题：

> **"我的项目里，最有可能遇到困难的地方是什么？"**

诚实地写下答案。 这个答案**就是你接下来要重点学的地方。**

卡住了怎么办

现象	原因	怎么办
想不出点子	正常，别硬想	**从"你最近抱怨过什么"开始找**（抱怨就是需求）
想法太大	最常见的错误	**砍到"一周能做完"的规模**（见下节）
数据模型想不出来	缺练习	照着记账簿的 7 张表**模仿结构**
不确定技术选型	正常	**先按 6.2 的三个问题走**，别纠结细节
觉得自己做不出来	每人都这么想	**见下面的"最小可用版本"**

八、最小可用版本（MVP）：决定你能不能做完

8.1 一个真实的数据

大部分个人项目失败的原因是"做得太大"，不是"技术不够"。

对比一下：

	太大	**最小可用**
范围	记账 + 预算 + 报表 + 云同步 + 手机端	**能记一笔、能看到列表**
完成时间	永远做不完	**一个周末**
结果	半成品烂尾	**能用，然后持续加功能**

记账簿本身就是这么长出来的——回顾它的开发过程（这是真实的）：

```
第一版：记一笔 + 列表 + 账户分类
第二版：日历、周期记账、附件
第三版：动效、图标重设计
第四版：截图识别、预算、汇总、搜索
第五版：应用锁、导入导出、备份、发布
```

每一版都是"能用的完整产品"，而不是"做了一半的更大产品"。

8.2 怎么砍到最小

方法：把所有需求按"没有它就没有意义"排序，只留前 1–2 条。

```
你的需求列表：
① 能记录数据          ← 没有它，这就不是软件   ✅ 留
② 能看到记录列表      ← 没有它，记了也看不到   ✅ 留
③ 能修改记录          ← 重要，但可以下一版
④ 能搜索              ← 重要，但可以下一版
⑤ 数据统计图表        ← 下一版
⑥ 深色模式            ← 下一版
⑦ 云同步              ← 很后面
⑧ 手机端              ← 很后面

结论：第一版只做 ① 和 ②
```

为什么"只做两条"反而更快做完：因为你会在**做 ① ② 的过程中**发现很多没想到的细节（数据怎么校验、列表怎么排序、空状态显示什么）。**这些细节才是真正花时间的地方。**

8.3 一条给你的建议

> **第一个自己的软件，目标不是"做出好东西"，而是"完整走完一遍流程"。**

因为"完整走完一遍"（从想法到打包到能用）本身就极有价值：

- 你会知道**每个环节大概花多久**
- 你会知道**哪些地方容易卡住**
- 你会有一套**自己的项目骨架**（下次直接复用）

第二个软件会比第一个快得多——因为流程你已经熟了。

九、自测题

1. 四步设计法的顺序是什么？为什么不能从界面开始想？

2. 为什么"金额要用整数存分"这条经验适用于所有涉及钱的软件?

3. 什么时候该自己做组件（而不是用现成的库）?

4. 什么是"最小可用版本"?为什么它能决定你会不会做完?

5. "第一个自己的软件，目标不是做出好东西"——这句话是什么意思?

答案与解析

1. ① **需求定义** → ② **数据模型** → ③ **界面与流程** → ④ **技术选型。** → 不能从界面开始，因为**数据模型是地基。**先想界面，做到一半会发现"存不下想要的字段"，那时返工代价很大。

2. 因为计算机用二进制表示小数会有**精度误差**（`8.7 * 100 = 869.9999999999999`）。 → 任何涉及**钱、精确计数、需要累加**的场景，都要避开浮点数。**这是计算机的固有问题，和用哪种语言无关。**

3. ① **需求很简单，而现成的库很大**（比如记账簿自己写 xlsx 写出器，因为只需要"一张表 + 表头加粗"）；② **这是产品的核心差异化**（比如界面和动效，要精确控制）。 → 反之：通用、成熟、无特殊要求的（如 OCR、图表），**直接用现成的。**

4. 只保留"没有它这个软件就没意义"的 1–2 条需求，**一个周末就能做完的第一版。** → 因为**大部分个人项目失败在做太大。**MVP 让你**先有一个能用的东西**，然后在它基础上持续加功能（记账簿就是这样长了五版）。

5. 意思是：**第一版的重点是"完整走完一遍流程"**（从想法 → 设计 → 实现 → 打包 → 能用），而不是追求质量。 → 因为**走完一遍本身价值极大**：你会知道每个环节花多久、哪里容易卡、并且得到一套可复用的项目骨架。**第二个软件会快得多。**

十、术语表

术语	一句话解释
需求定义	用可验证的语言说清"要做什么、不做什么"
验收标准	"怎么算做完了"的判断依据
数据模型	要存哪些表、哪些字段、什么关系
字段类型	每个字段存什么形式的数据（文字/整数/时间）
软删除	不真删，只标记删除时间
业务日期 vs 时间戳	用户视角的日期 vs 系统记录的精确时刻
以任务为中心	从"用户要完成什么"设计界面，而非"要几个页面"
空状态	还没有数据时界面显示什么
技术选型	决定用什么技术实现
MVP / 最小可用版本	只保留核心功能、能立刻用起来的第一版
范围蔓延	需求在开发过程中不断变大

十一、本讲与项目的对应

- 你产出的东西：**一份自己的软件设计方案**（需求 + 数据模型 + 流程 + 选型）
- 我建议你**把它写成一个文本文件保存**——下一讲要用
- **本讲不动任何代码**
- 上一讲：6-2 发布 ｜ 下一讲：**7-2 把它做出来并交付**（最后一讲）

讲义版本：v1 ｜ 对应项目版本：0.1.0